Advanced Databases - Assignment 3

College project for Advanced Databases at U-TAD

Authors:

- Emilie Dubief
- Itziar Morales Rodríguez

Toolset:

- Python (3.12)
- MongoDB
- Redis
- Nix (dependencies)
- LaTeX (documentation)

Project structure

- The necessary files for the ODM are in the `ODM/` folder.
- `data/` contains mock data for users, companies and educational centers plus the model schemas.
- `queries.ipynb` is the jupyter notebook that runs a series of queries
- `nix.shell` only needed to install necessary python dependencies
- `doc.pdf` contains a brief explanation of the project

```
project_root/
|-- .git/
|-- data/
|   |-- users.json
|   |-- companies.json
|   |-- educational_centers.json
|   |-- models.yml
|-- environment.yml
|-- help_desk.py
|-- send_request.py
|-- session_help_desk_test.py
|-- sessions.py
|-- queries.ipynb
|-- README.md
|-- shell.nix
|-- ODM/
|   |-- __init__.py
|   |-- app.py
|   |-- geo.py
|   |-- model.py
```

Project definition

ODM

This project includes an ODM layer (in `ODM/model.py`) that connects to MongoDB and provides model classes. The ODM also uses Redis as a caching layer for certain read operations (see the short summary below).

Session

For the session part, we needed to store the session data and the token.

The session data is composed of:

- username
- fullname
- password
- privileges

All this attributes will be stored as string in redis to avoid using too many types of data. The key used for the session will be user:username, this will create unique keys because two users won't be able to have the same username. The attributes od the session will be stored in a hash in redis with the following template: "user:username" {"username":"username", "fullname":"fullname", "password":"password", "privileges":"privileges"}

The token will be stored independently because it needs to have an expiration date of one month. The token value will appear in the name of the key and the value will be the username of the session linked to the token. It will be stored in redis with the following template: "token:token_value" "username"

HelpDesk

For the helpdesk, we needed to store the help requests (username, priority and message) and the list of help requests

The list of help requests is called "HelpRequests", its values are the ids of help requests, they are composed like this: priority:token

The help requests are stored individually as hashes composed of: priority:token {"username":"username", "message":"message"}

When a help request has been read and deleted in the list, the hash related is deleted too.

To test this functionment, you need to launch the main.py in a terminal and the send_request.py in another terminal. This allows you to test the functionment of the waiting of help requests.

Redis caching in the ODM

Initialization

The Redis cache gets initialized in the `init_class(...)` function due to it needing to be accessed by each `Model` class at class-level.

init_class(...)

- Creates a class-level Redis client at `cls._redis_client`.
- Configures Redis: maxmemory = 150MB and policy = volatile-ttl.
- Purpose: one shared client per model class and bounded memory use.

Find and aggregate functions

The find and aggregate functions in the `Model` class cache the query result in cache, and use the cache to access them if they exist in cache already.

Per-class keys are used to avoid collisions across models. Examples:

- `<Model Name>:byid:<ID string>`
- `<Model Name>:find:<Filter hash>`
- `<Model Name>:aggregate:<Pipeline hash>`

MongoJSONEncoder converts ObjectId to string before JSON storage, so cached values are plain JSON strings. These cached values use cached keys set to `ex=86400` to bound lifetime and allow eviction.

find(filter)

- Cache key: `<Class>:find:<filter hash>`.
- If cached JSON exists, return it (wrapped in a ModelCursor).
- On miss, query MongoDB, cache JSON with ex=86400 (24h), return results.

aggregate(pipeline)

- Cache key: `<Class>:aggregate:<pipeline hash>`.
- Mirrors find(): check cache, run aggregation if miss, cache with ex=86400.

find_by_id(id)

- Cache key: `<{Class}:byid:<id>` where id is stringified.
- If cached, instantiate Model from JSON and return it.
- On miss, read from DB and set cache with `ex=86400`.

Save and delete functions

In both the `save` and `delete` functions the queries for `find` and `aggregate` need to be delete from cache due to risk of data not being up to date with the DB. This is done using `SCAN` to find and delete keys matching:

- `"_:find:_"` and `"_:aggregate:_"`.

For `find_by_id` this doesn't need to be done since both `save` and `delete` modify the cache for the specified document accordingly.

save(self)

- After insert/update, write-through updates by-id cache with the current document JSON (`ex=86400`).
- Calls `_clear_find_and_aggregate_cache()` to invalidate query/agg caches.

delete(self)

- After DB delete, remove the by-id key (if present) and call `_clear_find_and_aggregate_cache()`.

Dependencies

If you don't have the Python dependencies installed locally, you can get them through the shell.nix file with the following commands:

Conda

If you have conda, you can install the dependencies using:

```
# Create conda environment from YML file
conda env create -f environment.yml
conda activate ODMEmilieItziarRedis
```

In case the wrong kernel is used, run the following command to get the right kernel to be used (change Kernel in Jupyter to the proper one)

```
python -m ipykernel install --user --name ODMEmilieItziarRedis \
  --display-name "Python (ODMEmilieItziarRedis)"
```

Nix

Using the nix package manager if you don't have conda:

```
# Install nix (authenticate with sudo)
sh <(curl --proto '=https' --tlsv1.2 -L \
  https://nixos.org/nix/install) --daemon
```

```
# Initialize nix-shell environment using shell.nix
nix-shell

# Run jupyter notebook
jupyter lab
```

How to Run

- Make sure you have `mongodb` running as a service in your system.
- Make sure you have the dependencies installed.
- Make sure you have `jupyter lab` or `jupyter notebook` running.

```
jupyter lab
```

Issues

Jupyter

Sometimes the jupyter notebook doesn't render stuff properly, re run all cells whenever this happens.

ODM

`ODM.getGeoLocation` can return a timed out error due to API but cannot be changed due to the test.

Session

`Session.create` can return two exceptions:

- an Exception if a key is missing between username, fullname, password and privileges
- an Exception if the username for the new session is already used by another session

HelpDesk